HDOC DAY – 58

Algorithm

1. Start.
2. Read the size of the array.
3. Declare a variable-length integer array arr[size].
4. Read the array elements.
5. Read the value of k.
6. Pass the array address, size, and k to kth_smallest().
7. Check whether k is valid. If k <= 0 or k > size, display -1.
8. Sort the array in ascending order using Bubble Sort.
9. Find the element at index k - 1 in the sorted array.
10. Display it as the kth smallest element.
11. Stop.

C PROGRAM:

```
#include <stdio.h>


void kth_smallest(int *arr, int size, int k)
{
   int i, j, temp;


   if (k <= 0 || k > size)
   {
      printf("-1\n");
      return;
   }


   // Bubble Sort
```

```
    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    printf("kth smallest element: %d\n", *(arr + k - 1));
}

int main()
{
    int size, i, k;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter the array elements: ");
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);
```

```
    printf("Enter k: ");
    scanf("%d", &k);

    kth_smallest(arr, size, k);

    return 0;
}
```

TEST CASE TABLE:

Test Case	Array	k	Expected Output
1	7 10 4 3 20 15	3	kth smallest element: 7
2	7 10 4 3 20 15	4	kth smallest element: 10
3	7 10 4 3 20 15	7	-1
4	4 2 2 7 5	3	kth smallest element: 4
5	5 1 3 2 4	1	kth smallest element: 1